East West University

Department of Computer Science and Engineering

CSE110: Mini Project Specification [Fall 2025]

Smart Task Manager

Overview

Design and implement a small interactive Java application that demonstrates core object-oriented programming concepts: class, object, encapsulation, inheritance, polymorphism, and exception handling. The program will be a console-based task manager that lets a user add tasks, mark tasks complete, and view tasks in different ways. Keep the program simple and easy to use.

Learning Objectives

- Define and instantiate Java classes and objects.
- Apply encapsulation with private fields and public getters/setters.
- Implement inheritance and create subclasses that extend a base class.
- Use polymorphism (store different subclass objects in a single collection and call overridden methods).
- Handle errors robustly using try–catch and a custom exception.
- Build a menu-driven interactive console application.

Deliverables

- All `.java` source files.
- A project report (Word) describing design, implementation, testing, and conclusions (template provided earlier).

Functional Requirements

1. ***Menu-driven Console Interface***
   - Present a clear numbered menu and repeat until the user chooses to exit.
   - Accept input via `Scanner`.
   - Handle invalid menu input (non-numeric or out-of-range) using exception handling.
2. ***Add a New Task***
   - Prompt for `title` and `description`.
   - Ask for task type: `1` = WorkTask, `2` = PersonalTask.
   - Create an instance of the appropriate subclass and store it in an `ArrayList<Task>`.
   - Confirm that the task was added.
3. ***Mark an Existing Task Completed***

- Show an indexed list of tasks.
- Prompt the user for the index to mark completed.
- Validate index; if invalid, throw/handle **`TaskNotFoundException`**.
- Set **`isCompleted`** to **`true`** and confirm.

4. ***Display All Tasks***
   - Print each task using a polymorphic method (e.g., overridden **`getDetails()`** or **`toString()`**).
5. ***Display Completed Tasks***
   - Filter and print tasks where **`isCompleted == true`**.
   - Show a friendly message if none exist.
6. ***Display Pending Tasks***
   - Filter and print tasks where **`isCompleted == false`**.
   - Show a friendly message if none exist.
7. ***Data Storage (In-Memory)***
   - Maintain all tasks in a single collection: **`ArrayList<Task>`**.
   - No file I/O required.
8. ***Exception Handling***
   - Use try–catch for common input errors (e.g., **`NumberFormatException`**, **`InputMismatchException`**).
   - Implement at least one custom exception: **`TaskNotFoundException`**.
   - Ensure the program never crashes from invalid user input.
9. ***User Feedback***
   - Show confirmation messages and clear error messages.
   - Keep console output readable and well formatted.
10. ***Code Quality***
    - Use private fields and public getters/setters.
    - Provide meaningful method and class names.
    - Comment key methods and classes.

Non-functional Requirements

- Code should compile and run on Java 8+.
- Provide readable, well-commented source.
- Keep UI minimal (console).
- No use of advanced Java GUI or external libraries.

Optional Extensions

- Add file persistence (save/load tasks).
- Add due dates (use **`java.time.LocalDate`**) and sort tasks by due date.
- Add priority levels.
- Add search/filter by keyword.
- Add ability to edit or delete tasks.

Grading Criteria

- Correct usage of OOP concepts (classes, encapsulation, inheritance, polymorphism).
- Working menu and correct functionality.
- Robust exception handling and input validation.
- Code readability, comments, and style.
- Quality of the project report.

Submission Deadline

- TBA.

Interview / Viva Date

- TBA.

Marking Rubric

Component	Marks (out of 10)
Program Code (Java source files)	6
Project Report (DOCX)	2
Individual Interview (VIVA)	2